

Programación de Sistemas

Práctica 01: Extensión del analizador léxico para IMP++

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

Descripción

Extiendan el analizador léxico funcional de IMP desarrollado en clase para reconocer las construcciones de IMP++. Deberán modificar sus reglas léxicas, conservar la compatibilidad con IMP y comprobar el resultado con pruebas. Esta versión será la entrada de la Práctica 2.

La práctica se realiza en equipos de una a tres personas. Cada equipo entregará su trabajo mediante un repositorio privado compartido.

Objetivos

1. Extender la especificación léxica de IMP con los tokens de IMP++.
2. Resolver conflictos entre palabras reservadas, identificadores y operadores compuestos.
3. Verificar la extensión sin romper los programas válidos de IMP.

Desarrollo

Ejercicio 1. Creen el repositorio del equipo.

- Formen un equipo de una a tres personas y acuerden un nombre para identificarlo.
- Una persona del equipo debe crear en GITHUB un repositorio privado para las cuatro prácticas.
- Añadan como colaboradores a todas las personas del equipo y al usuario manu-msr.
- Registren en el README.md el nombre del equipo, los nombres de sus integrantes y sus usuarios de GITHUB.
- Copien la carpeta `scripts_estudiante` al repositorio y renómbrenla como `practica1_lexer_imp_mas_mas`.
- Al terminar, el acceso registrado en GITHUB debe cumplir:

Visibilidad: `privada`

Equipo: `1 a 3 integrantes con acceso`

manu-msr: `invitación pendiente o colaborador`

- Para entregar, una persona del equipo debe pegar la liga al repositorio en los comentarios de la actividad en GOOGLE CLASSROOM.

Ejercicio 2. Verifiquen el punto de partida.

- Preparen el entorno con las instrucciones del README.md; todavía no modifiquen el lexer.
- Ejecuten el ejemplo base y las pruebas:

```
python lexer.py ejemplos/imp_base.imp
python -m unittest -v
```

- Deben aparecer 14 tokens. Ejemplo de salida:

```
LOC 'X' 1:1
ASSIGN ':= ' 1:3
NUM '0' 1:6
SEMI ';' 1:7
...
WHILE 'while' 2:1
...
Ran 10 tests in ...
OK (skipped=5)
```

- Si el inicio, la posición de `while` o el resumen no coinciden, revisen la instalación o la copia de los archivos.

Ejercicio 3. Extiendan identificadores y comentarios.

- Abran `imp.lark` y modifiquen `LOC` con el patrón `[A-Za-z][A-Za-z0-9_]*`. Comprueben que `contador_1` se reconozca como un solo `LOC`.
- Definan `COMMENT` para reconocer desde `//` hasta el final de la línea y agreguen la instrucción `%ignore COMMENT`.
- Retiren los decoradores `@unittest.skip` de las pruebas de identificadores y comentarios.
- Ejecuten las pruebas:

```
python -m unittest -v
```

- Ejemplo de salida:

```
test_comentario_se_ignora ... ok
test_identificador_con_guion_bajo ... ok
...
Ran 10 tests in ...
OK (skipped=3)
```

- Estas pruebas revisan que `contador_1` produzca un solo `LOC`, que el comentario no produzca tokens y que `Y` conserve la posición línea 2, columna 1.

Ejercicio 4. Agreguen palabras y símbolos de IMP++.

- Agreguen en `imp.lark` las reglas para `int`, `bool`, `print` y `for`. Conserve los nombres de token indicados en el archivo.
- Usen la misma prioridad y el mismo límite léxico de las palabras reservadas de IMP.
- Agreguen las reglas para las llaves y para los operadores `++` y `--`.
- Retiren los decoradores `@unittest.skip` restantes y ejecuten las pruebas:

```
python -m unittest -v
```

- Ejemplo de salida:

```
test_incremento_y_decremento_son_tokens_completos ... ok
test_palabras_reservadas_y_llaves ... ok
test_programa_imp_mas_mas ... ok
...
Ran 10 tests in ...
OK
```

- Las pruebas comparan, entre otros casos, `print printable` con `PRINT`, `LOC` y `X++ + Y-- - 1` con `LOC`, `INC`, `PLUS`, `LOC`, `DEC`, `MINUS`, `NUM`.

Ejercicio 5. Comprueben la compatibilidad y documenten.

- Ejecuten el ejemplo completo y todas las pruebas:

```
python lexer.py ejemplos/imp_mas_mas.imp
python lexer.py ejemplos/error_lexico.imp
python -m unittest -v
```

- `imp_mas_mas.imp` debe producir 30 tokens y no debe mostrar `COMMENT`. Ejemplo de salida:

```
INT 'int' 1:1
LOC 'contador_1' 1:5
ASSIGN ':= ' 1:16
NUM '0' 1:19
SEMI ';' 1:20
...
PRINT 'print' 5:5
LPAR '(' 5:10
LOC 'contador_1' 5:11
RPAR ')' 5:21
SEMI ';' 5:22
RBRACE '}' 6:1
```

- Ejemplo de salida para el archivo con error:

```
Error léxico en línea 2, columna 8: '@'
```

- Agreguen una prueba que combine al menos dos elementos nuevos.
- Agreguen otra para un carácter no reconocido y comprueben que produzca `UnexpectedCharacters`.
- Ejecuten nuevamente las pruebas. Ejemplo del resumen después de agregar sólo los dos casos solicitados:

```
...
Ran 12 tests in ...
OK
```

- Completen el `README.md` con el resultado, una decisión léxica y el tiempo real de trabajo.
- Entreguen la carpeta sin pruebas omitidas ni tareas `TODO` pendientes.

Referencias de consulta

- [1] Winskel, G. (1993). *The Formal Semantics of Programming Languages: An Introduction*, sección 2.1. The MIT Press.
- [2] Lark. *Grammar Reference y API Reference*. <https://lark-parser.readthedocs.io/>